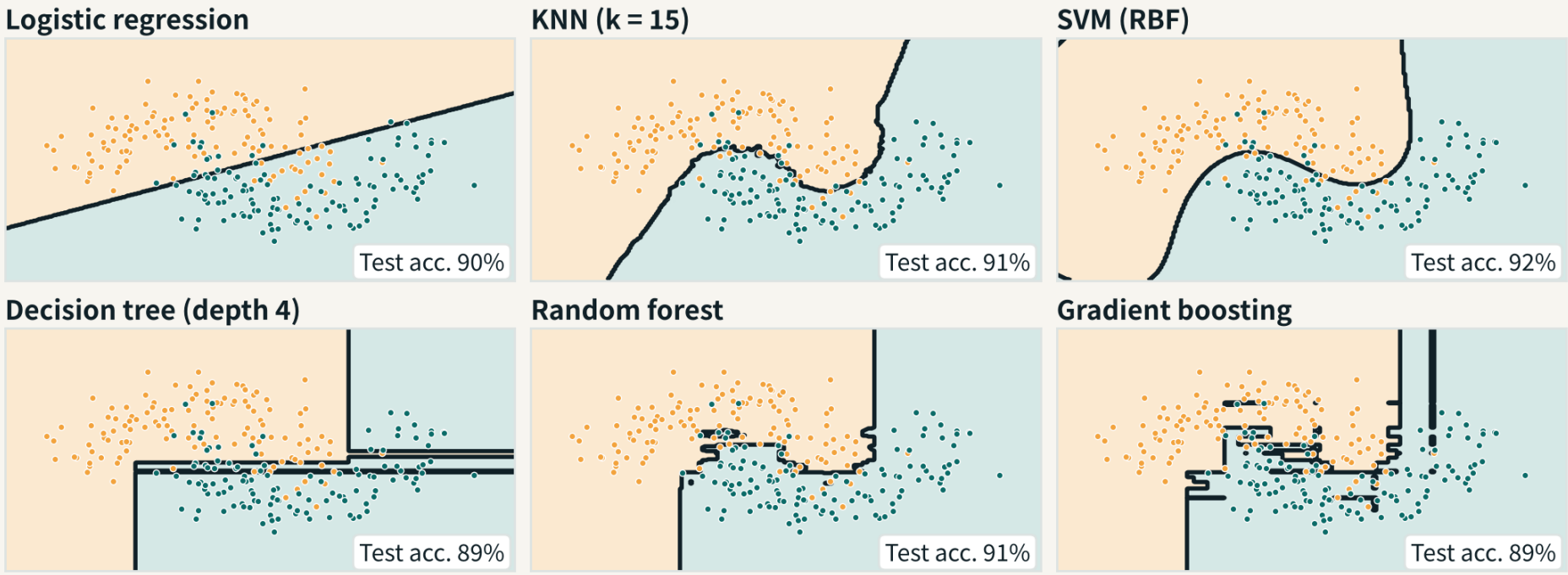
Same data, different boundary shapes for each model

Synthetic two-moons data (400 points), 60% train / 40% test · background = predicted class



Straight line (logistic) · smooth curve (KNN, SVM) · steps (tree) · combined steps (ensembles)

Accuracy looks similar, but boundary shapes differ: match the model to the data

※ Decision boundary: the line where the model switches class · Classification: predicting one of a fixed set of categories

Logistic regression splits classes with one straight line

It turns a score into a probability; despite the name, it is a classifier

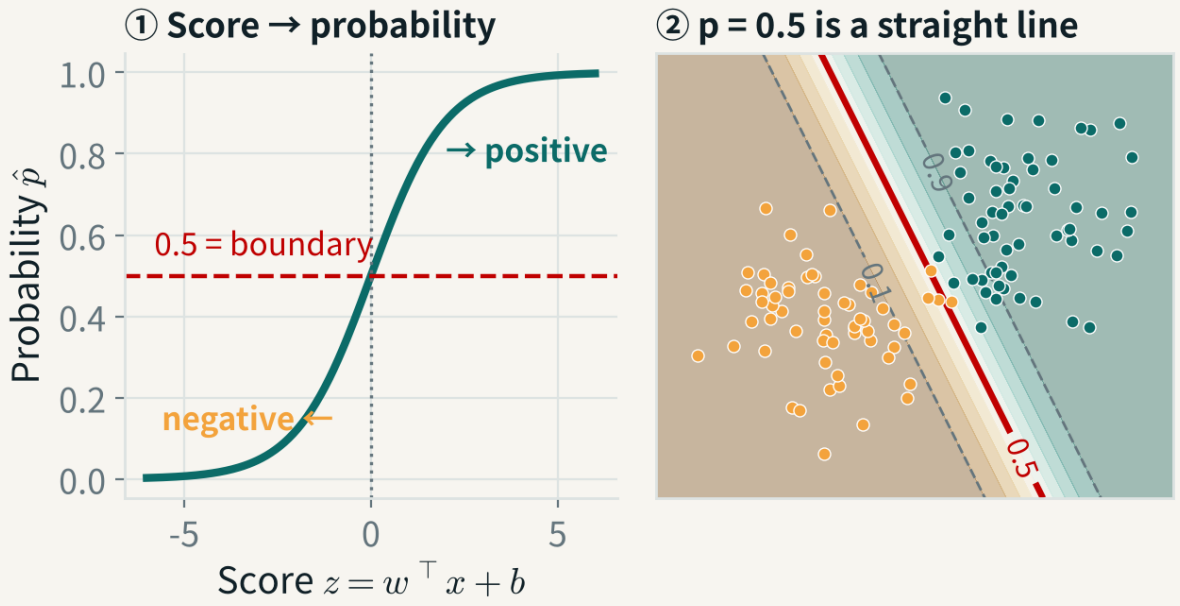
Sigmoid · squashes the score z into 0–1

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z = w_1 x_1 + w_2 x_2 + b$$

Log-odds · probability mapped back to a score

$$\log \frac{\hat{p}}{1 - \hat{p}} = z = w_1 x_1 + w_2 x_2 + b$$

- w: each feature's pull on the score
- Probability ≥ 0.5 → positive
- The boundary is always straight



Fast and interpretable through its coefficients: the standard baseline

Its straight boundary cannot capture curved patterns

KNN classifies by a majority vote of the k nearest neighbors

No real training: it memorizes the data and measures distances when a new point arrives

Distance · Euclidean distance

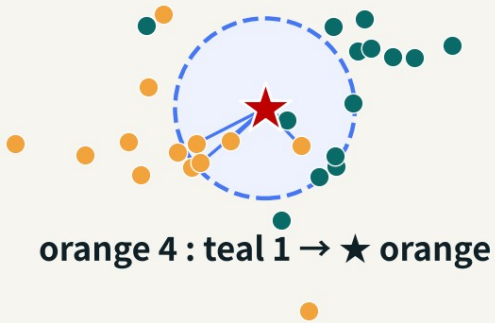
$$d(x, x') = \sqrt{\sum_j (x_j - x'_j)^2}$$

Prediction · majority label of k neighbors

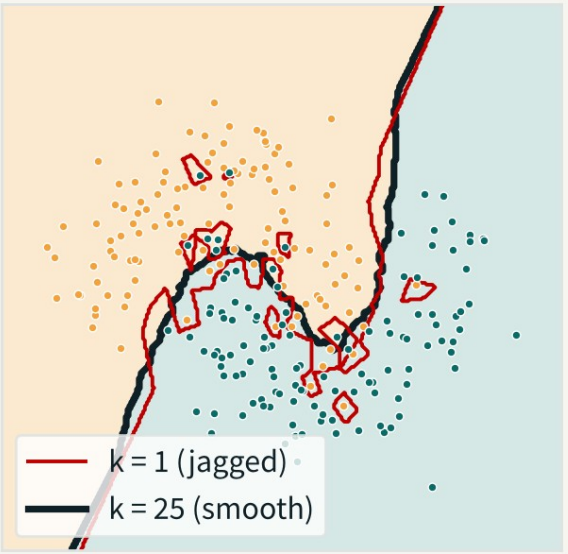
$$\hat{y} = \text{mode}\{\, y_i : i \in N_k(x) \,\}$$

- Small k: jagged (overfitting)
- Large k: too smooth (underfitting)
- Put features on one scale first

① ★'s 5 neighbors vote



② k sets the boundary shape



Choose k by cross-validation and always standardize features

With large data, every prediction must compute all distances, so it gets slow

SVM finds the boundary with the widest margin between the classes

Only the points closest to the boundary (support vectors) decide where it goes

Maximum margin

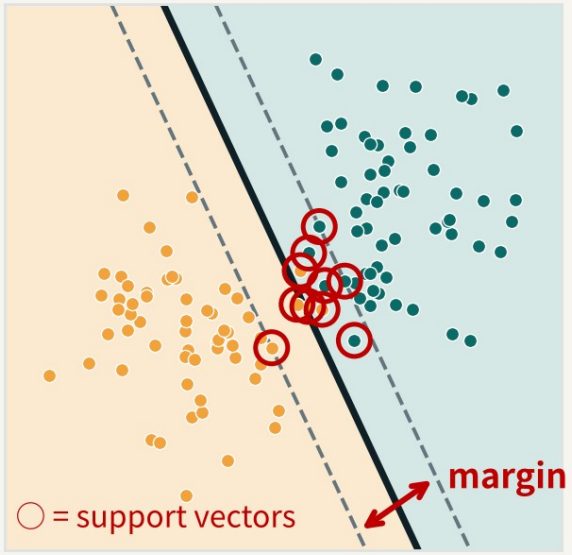
$$\max_{w,b} \frac{2}{\|w\|} \quad \text{s.t.} \quad y_i\,(w^\top x_i + b) \geq 1$$

RBF kernel · similarity close to 1 for nearby points

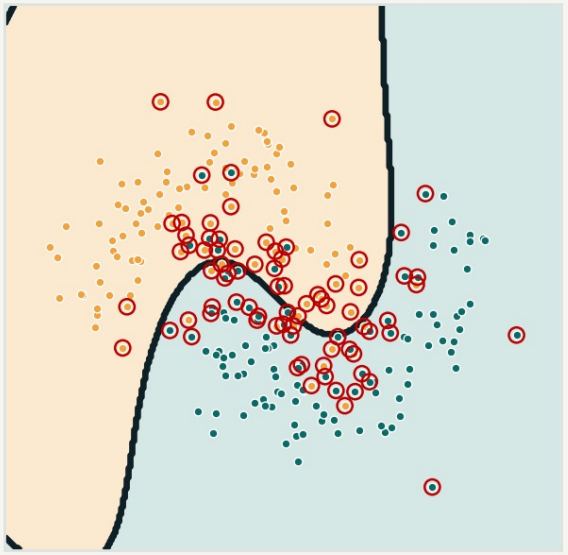
$$K(x, x') = \exp\left(-\gamma \,\|x - x'\|^2\right)$$

- Wider margin → better on new data
- C: how many mistakes to tolerate
- Kernel: lets the boundary curve

① The widest-margin line



② Kernel (RBF) bends the line



Accurate on mid-sized data; kernels handle straight and curved boundaries

Slow on large data; needs tuning of C and γ plus standardization

A decision tree chains yes/no questions and carves the space into boxes

Each question picks the feature and cut-off that most reduce impurity (Gini)

Gini impurity · 0 means a pure node

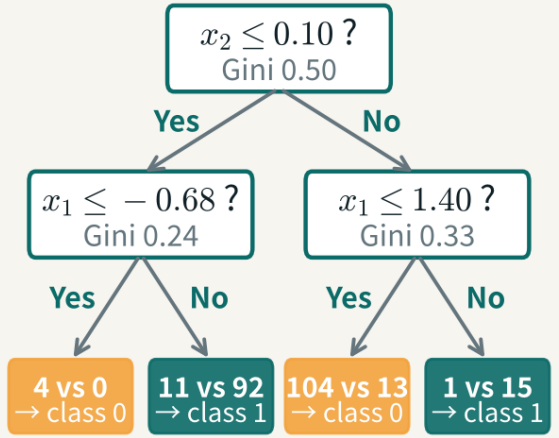
$$G = 1 - \sum_k p_k^2$$

Example · a node with 8 orange and 2 teal

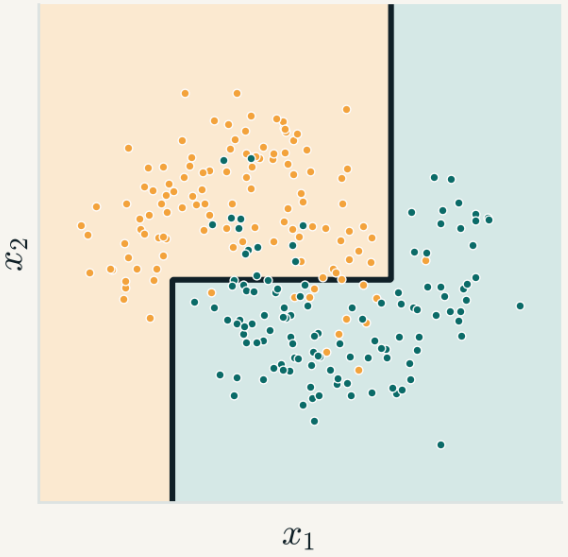
$$G = 1 - \left(0.8^2 + 0.2^2\right) = 0.32$$

- Question = one feature + a cut-off
- Deeper → more detail, more overfitting
- If-then rules are easy to explain

① A tree of yes/no questions



② Result: axis-aligned boxes



Readable as rules and needs almost no preprocessing

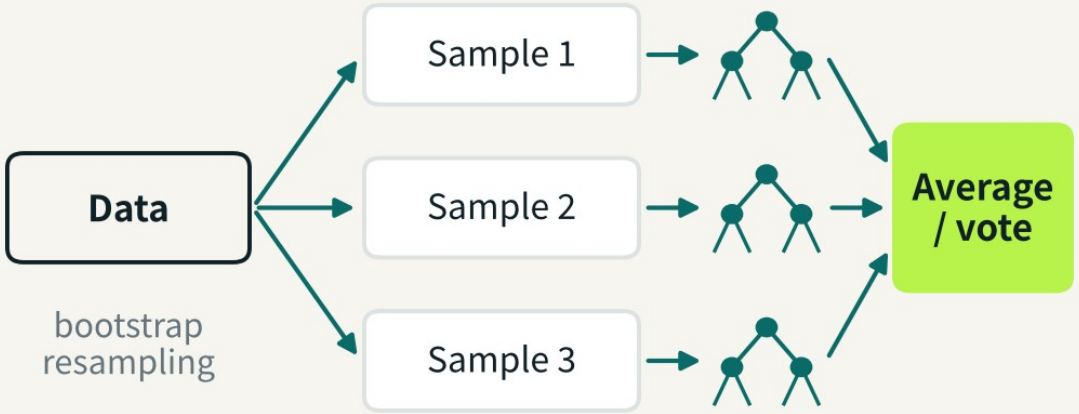
Unstable: a small change in data can reshape the whole tree

Bagging averages trees side by side; boosting fixes errors in sequence

Random forest is the flagship of bagging; gradient boosted trees (GBT) of boosting

① Bagging → Random forest

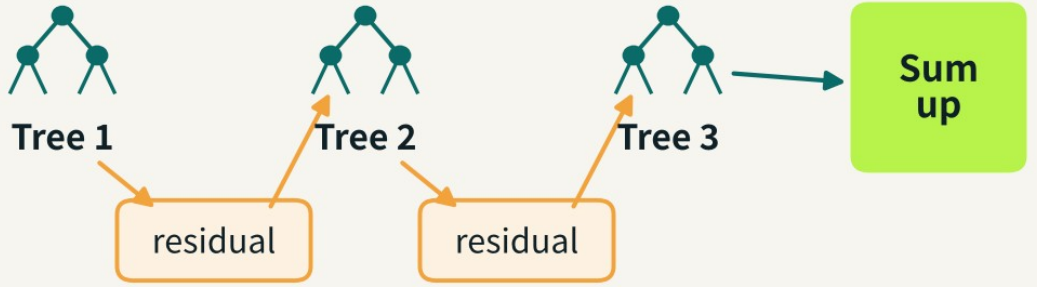
Parallel: trees are independent



$$\hat{f}(x) = \frac{1}{B}\sum_{b=1}^{B} f_b(x)$$

② Boosting → Gradient boosting

Sequential: each tree learns the previous error



small learning rate η corrects step by step

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

Bagging cuts variance (jitter); boosting cuts bias (systematic error)

Random forests are stable and easy to tune; boosting is often more accurate but needs tuning